《Java 企业级应用》课堂作业

ATM 案例分析

—— 基于继承与多态的设计与实现

第 5 章 继承与多态 · 案例研究

2026 年 10 月

目录

一、需求分析

1.1 项目背景与目标　1.2 功能需求　1.3 与课件版设计的主要差异

二、设计原则

2.1 分析路线　2.2 用例视图　2.3 识别类：从需求名词到类

2.4 静态结构：类图（银行层、交易家族、会话层与硬件层）

2.5 面向对象设计原则的落实

2.6 动态结构：活动图（认证、取款、存款、转账）

2.7 动态结构：时序图（取款、转账、认证）

2.8 状态建模：银行卡的一生　2.9 可见性设计

三、代码分析

3.1 工程结构与运行方式　3.2 继承与多态的应用

3.3 使用的设计模式　3.4 关键实现与设计的一致性

3.5 测试与验证　3.6 优点与缺点　3.7 改进方向

四、总结

附录 A 端到端会话记录（节选）　附录 B 编译、运行与测试

一、需求分析

1.1 项目背景与目标

本次课堂作业以第 5 章课件中的 ATM 案例为素材。课件版给出了一套“12 个类、三类交易”的完整示例设计；但作业要求我们按照自己的思路重新拆解 ATM——允许创造新的类和新的时序图——并给出可运行的代码实现。因此我们重新走了一遍从需求到代码的完整过程，本文档即为该过程的完整记录。

本设计的目标有三个：

- 功能完整：覆盖认证、查询余额、查询交易流水、取款、存款、转账六个用例，并实现真实 ATM 的关键行为——密码连续输错三次吞卡、取款限额、钞箱盘点。
- 设计自洽：从需求出发识别类、属性、操作与协作，用用例图、类图、活动图、时序图、状态图完整表达设计，图纸与代码一一对应、可以互相印证。
- 知识落地：把第 5 章的继承、多态、抽象类、接口、泛型、密封类（sealed）、记录类（record）、枚举等知识点真正用在设计里，每一处使用都有明确理由。

1.2 功能需求

系统模拟一台控制台版 ATM 机：用户在控制台完成“插卡 → 输入密码 → 办理交易 → 退卡”的完整流程。系统支持的六个用例及其需求描述如下表。

用例	前置条件	主流程	异常 / 分支	后置结果
认证登录	已插入本行有效银行卡	输入密码→校验通过→显示主菜单	密码错误可重输（最多 3 次）；第 3 次错误吞卡；非本行卡、已吞卡拒收	建立本次会话
查询余额	已认证	显示可用余额与总余额	无	打印凭条；余额不变
查询交易流水	已认证	列出当前账户最近 10 条流水	无记录时给出提示	打印凭条
取款	已认证	选择金额→金额 / 余额 / 限额 / 钞箱四重校验→先记账、后吐钞	金额非法（非 100 整数倍）/ 余额不足 / 超出限额 / 现金不足 / 用户取消	余额减少、钞箱减少、记一条流水、打印凭条
存款	已认证	输入金额→确认存款槽收到信封→记账	金额非法 / 未收到信封 / 用户取消	余额增加、记一条流水、打印凭条
转账	已认证	输入目标账号与金额→校验→双方记账	目标账号无效（不存在或等于本账户）/ 余额不足 / 用户取消	双方余额变化、双方各记一条流水、打印凭条

非功能需求与约束：

- 纯 Java SE 控制台程序，不依赖任何第三方库；开发环境 JDK 21，代码使用 sealed、record、switch 表达式等 JDK 17+ 特性。
- 数据保存在内存中（预置三个演示账户），不做持久化——这是教学规模的简化，改进方向中给出了持久化方案。
- 金额以 double 表示（教学简化）；生产系统应使用 BigDecimal 或“分”为单位的整数，见 3.7 节。
- 六个硬件部件全部面向接口抽象，控制台实现仅用于教学模拟；接口抽象同时使自动化测试成为可能（用测试替身替换真实设备）。
- 所有提示为中文；键盘输入做健壮性处理（非法输入循环重试；输入流提前结束则安全退出，不抛异常堆栈）。

1.3 与课件版设计的主要差异

重新分析的过程中，我们保留了课件版“分层清晰、职责单一、只让交易族使用继承”的精髓，但在若干关键点上做了自己的取舍。下表列出主要差异及其理由——这些差异正是“自己的思路”的体现。

设计点	课件版	本设计	理由
卡与账户	直接输入账号数字	新增 BankCard（银行卡）与 CardReader（读卡器）	更接近真实设备；卡是有状态的实体（会吞卡），必须被系统记住
认证	校验不通过无限循环重输	最多 3 次，第 3 次吞卡	真实 ATM 行为；循环必须有出口，状态要落库
交易种类	余额 / 取款 / 存款 3 种	5 种：新增“交易流水”“转账”	转账是多账户协作的代表（即课件思考题 2 的实践版）
取款金额	6 个固定档位	5 个档位 + 自定义金额	兼顾便捷与灵活；自定义金额引出“100 整数倍”校验
限额控制	无	策略模式：普通 / VIP 双限额	真实银行规则；为“策略模式”提供真实载体
硬件部件	4 个具体类	6 个接口 + 控制台实现	面向接口使硬件可替换、可测试；新增凭条打印机
执行骨架	三个子类各自重写 execute()	execute() 为 final 模板方法，子类只写 perform()	记流水、打凭条等公共步骤不重复；钩子方法支持转账双流水
交易家族	普通继承	sealed 密封继承	家族封闭可控：新增交易必须显式声明，穷尽 switch 有编译器保证
结果表达	void / boolean	TransactionResult 记录类（状态、消息、金额、余额）	失败原因结构化；流水与凭条统一处理
主菜单	直接打印与读取	泛型 Menu<T> 组件	消除重复的菜单代码，选项值类型安全
交易流水	无	TransactionLog + TransactionRecord	支撑“查询流水”用例；同时是凭条的数据来源

二、设计原则

2.1 分析路线

我们采用与课件一致的面向对象分析路线：读需求 → 找名词 → 定类与关系 → 定属性 → 画活动图 → 定操作 → 画协作（时序图）→ 定可见性 → 编码验证。下面各节按这条路线逐一给出设计结果与理由。

2.2 用例视图

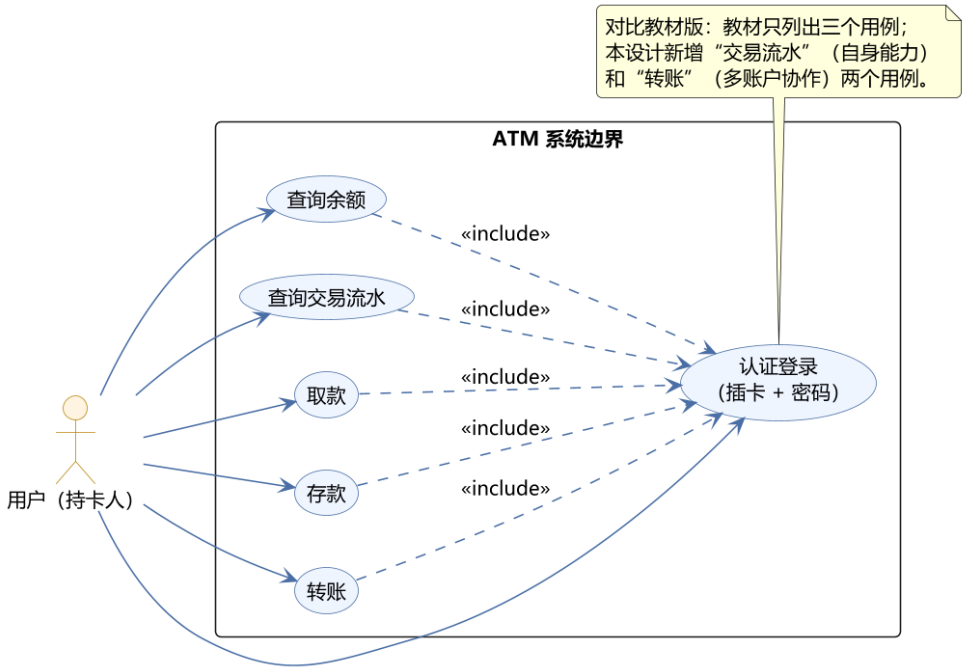


图 1　ATM 系统用例图

系统只有一个参与者——用户（持卡人）。六个用例中，“认证登录”是其余五个用例的公共前置条件，因此用 <<include>>（包含）关系表达：办理任何交易之前都必须先通过认证。与课件版相比新增了“查询交易流水”（账户自身的能力）与“转账”（多账户协作）两个用例；转账用例的加入直接引起了下文多处设计变化（目标账号校验、双边记账、双流水）。用例图只回答“系统提供什么功能”，不涉及内部实现，它是后面一切设计工作的对照基准。

2.3 识别类：从需求名词到类

把需求里的名词逐个列出，判断标准只有一条：系统需要为它保存数据、或者为它提供服务吗？需要，就是候选类；只是描述别人的，就是属性；系统范围之外的实体不建模。

需求名词	建模结果	理由
用户	参与者（Actor），不建模	他只是触发系统的人，系统不需要记住他
账户	类 Account	要保存余额，并按账户粒度记账
银行卡	类 BankCard	本设计新增：卡有状态（正常 / 已吞卡），会随认证失败迁移

账号、密码、余额、今日已取	Account 的属性	描述账户自身的数值
卡号、卡状态	BankCard 的属性	描述卡自身
钱	不建模	用 double 金额表示即可
银行	系统范围之外	BankDatabase 是“银行在本机的一个代理”，不是银行本身
取款机	类 ATM	系统本体（会话主控）
屏幕、小键盘、读卡器、出钞机、存款槽、凭条打印机	接口 + 控制台实现	它们是 ATM 的组成部分，但必须可替换、可测试
交易（取款 / 存款 / 转账 / 查询）	类族 Transaction + 5 个子类	它们是系统最主要的行为，各有独立规则
交易流水	记录类 TransactionRecord	需要保存并展示的结构化数据

2.4 静态结构：类图

2.4.1 银行数据层与限额策略

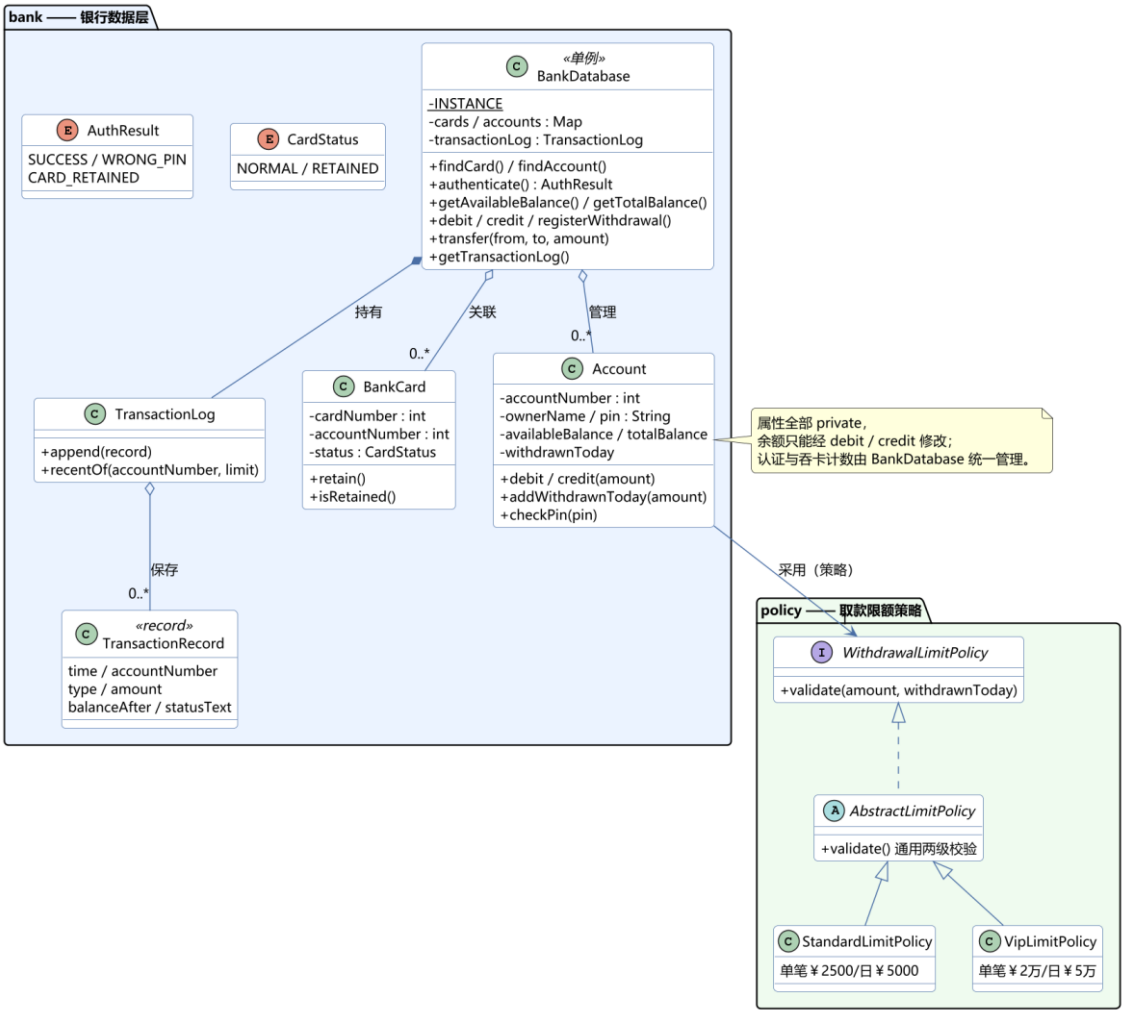


图 2　类图（一）：银行数据层与限额策略

类	职责	关键成员
BankDatabase	账户与卡数据的唯一入口（单例）；认证与吞卡计数也由它统一管理	authenticate、debit / credit、transfer、getTransactionLog

Account	一个银行账户：保存两种余额与当日已取金额；提供受控记账方法	debit / credit（仅有的余额修改入口）、checkPin
BankCard	一张银行卡：卡号、关联账户号、状态（正常 / 已吞卡）	retain、isRetained
TransactionLog	交易流水账本：追加记录、按账户查询最近记录	append、recentOf
TransactionRecord	一条流水（不可变记录类）：时间、账户、类型、金额、余额、结果	——
WithdrawalLimitPolicy	限额规则契约：单笔上限、当日累计上限、校验方法	validate
AbstractLimitPolicy	两级校验的公共骨架（抽象类）	validate（通用实现）
StandardLimitPolicy / VipLimitPolicy	普通 / VIP 两套限额数值	单笔 2500 / 日 5000；单笔 2 万 / 日 5 万

三个设计细节值得说明：其一，Account 的余额字段全部 private，唯一的修改途径是 debit / credit——封装让“记账”这一概念在代码层面不可绕过；其二，认证失败的计数不放在 ATM 上，而放在 BankDatabase 中，因为它属于“银行记住的事”而不是“一次会话记住的事”，这也让吞卡状态在会话之间持续有效；其三，账户通过组合持有一个限额策略对象，取款规则与账户数据分离。

2.4.2 交易家族：sealed 抽象父类与模板方法

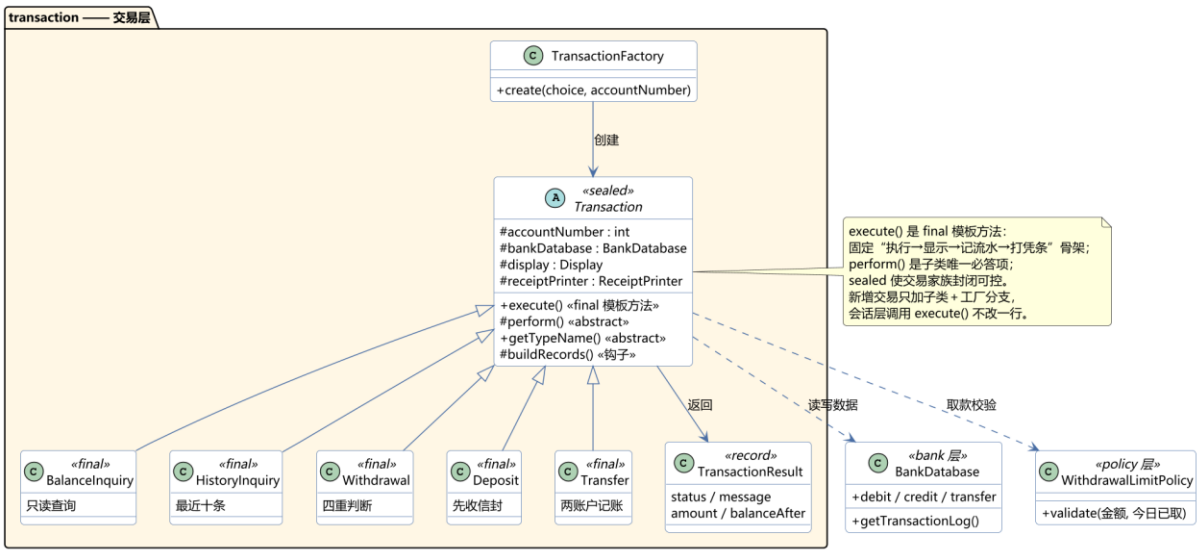


图 3　类图（二）：交易家族（sealed 抽象父类 + 模板方法）

交易家族是本设计使用继承的唯一位置，也是第 5 章方法论的集中体现：

- 向上抽取公共部分：账号、数据库、屏幕、凭条打印机等每个交易都需要的东西全部上提到抽象父类 Transaction；
- 向下保留变化点：perform() 是抽象方法，是子类的唯一必答项——五类交易各自实现自己的规则；
- 骨架固定：execute() 是 final 模板方法，把“执行 → 按结果展示 → 记流水 → 打凭条”这一骨架固化；子类无法（也不需要）改动它；

- 钩子方法：buildRecords() 默认生成一条本账户流水，转账子类覆写它生成两条（转出 + 转入）；
- 家族封闭：Transaction 声明为 sealed，permits 明确列出五个子类。交易家族是封闭的——将来必须新增交易时，编译器会强制你在 permits 名单中显式声明，避免随意扩散继承。

2.4.3 会话层与硬件抽象层

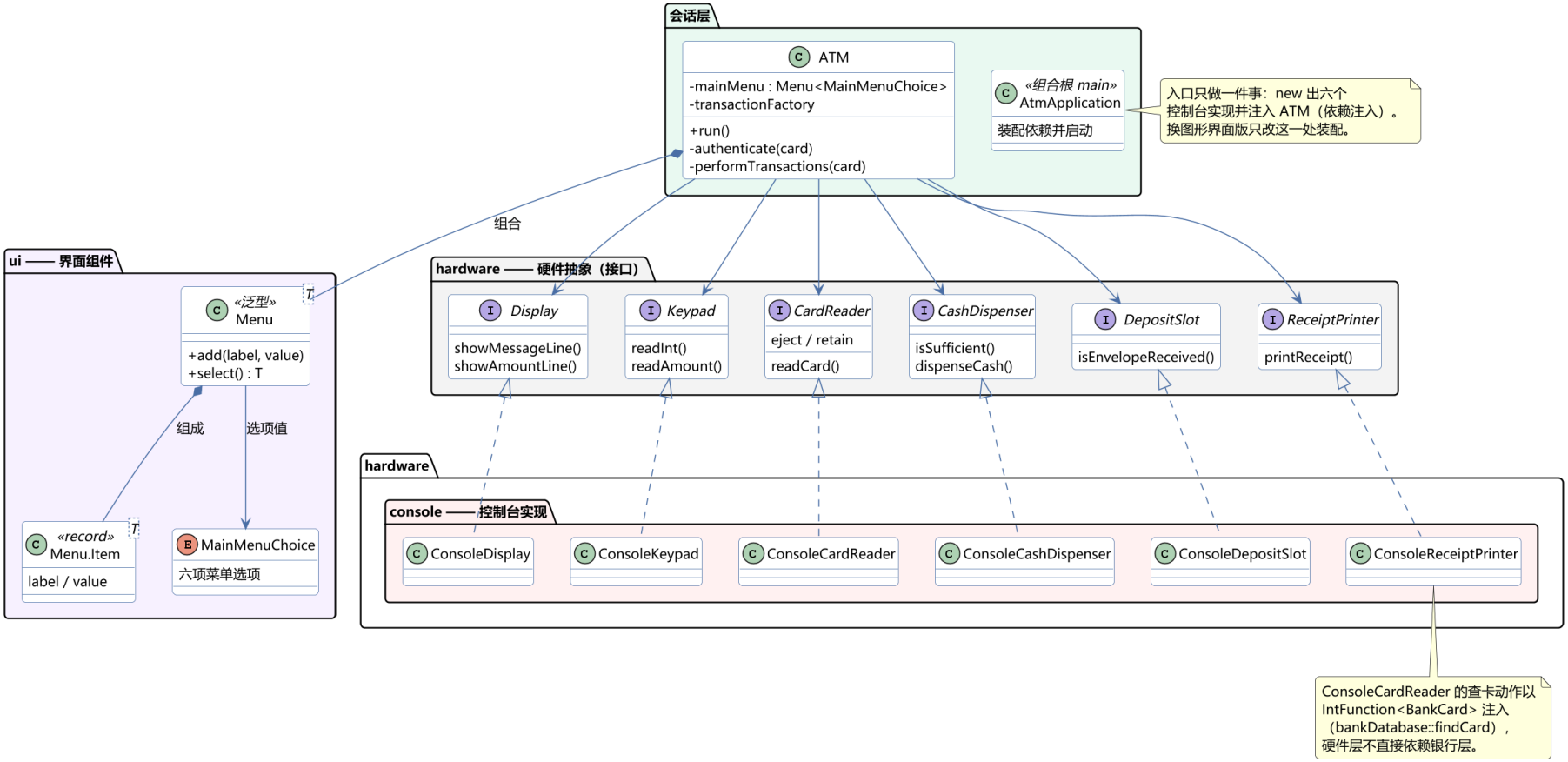


图 4　类图（三）：会话层与硬件抽象层（面向接口的六大部件）

会话层只有两个类：AtmApplication 是组合根——整个程序唯一负责“new 具体对象并注入依赖”的地方；ATM 只负责流程编排与会话状态（当前是哪张卡、认证是否通过），它不认识任何具体硬件，只认识六个硬件接口。硬件抽象层定义六个接口（屏幕、小键盘、读卡器、出钞机、存款槽、凭条打印机），hardware.console 包给出控制台实现。还有两个小细节：读卡器查询卡的动作以函数（IntFunction<BankCard>，即 bankDatabase::findCard）注入，因此硬件层不直接依赖银行层；ui 包的 Menu<T> 是泛型菜单组件，主菜单与取款菜单共用一个实现。

2.5 面向对象设计原则的落实

原则	一句话含义	本项目落点
单一职责	一个类只有一个变化的理由	ATM 管流程、交易子类管规则、BankDatabase 管数据、硬件类管输入输出——互不越界
封装	隐藏内部状态，只暴露安全的行为	Account 余额只能经 debit / credit 修改；钞箱张数私有，外部只问“够不够”
组合优于继承	只是复用能力就用“持有”，不是“是一种”就别继承	ATM 持有六个硬件接口；全程只有交易族使用继承
里氏替换	子类对象可以替换父类出现的位置	所有交易对象都能放到 execute() 的调用点上；子类不加重父类的约束（不会要求额外的前提条件）
开闭原则	对扩展开放，对修改封闭	新增交易＝新增子类 + 工厂加分支；新增硬件＝实现接口；既有交易、ATM、活动图均不修改
依赖倒置	高层模块依赖抽象而不是细节	交易与会话都依赖 Display / Keypad 等接口；具体控制台实现只在入口处装配（依赖注入）

2.6 动态结构：活动图

活动图回答“一次交易要经过哪些判断、按什么顺序”。四张活动图与代码中的方法一一对应：图里的每个菱形，就是 perform() 里的一个 if。

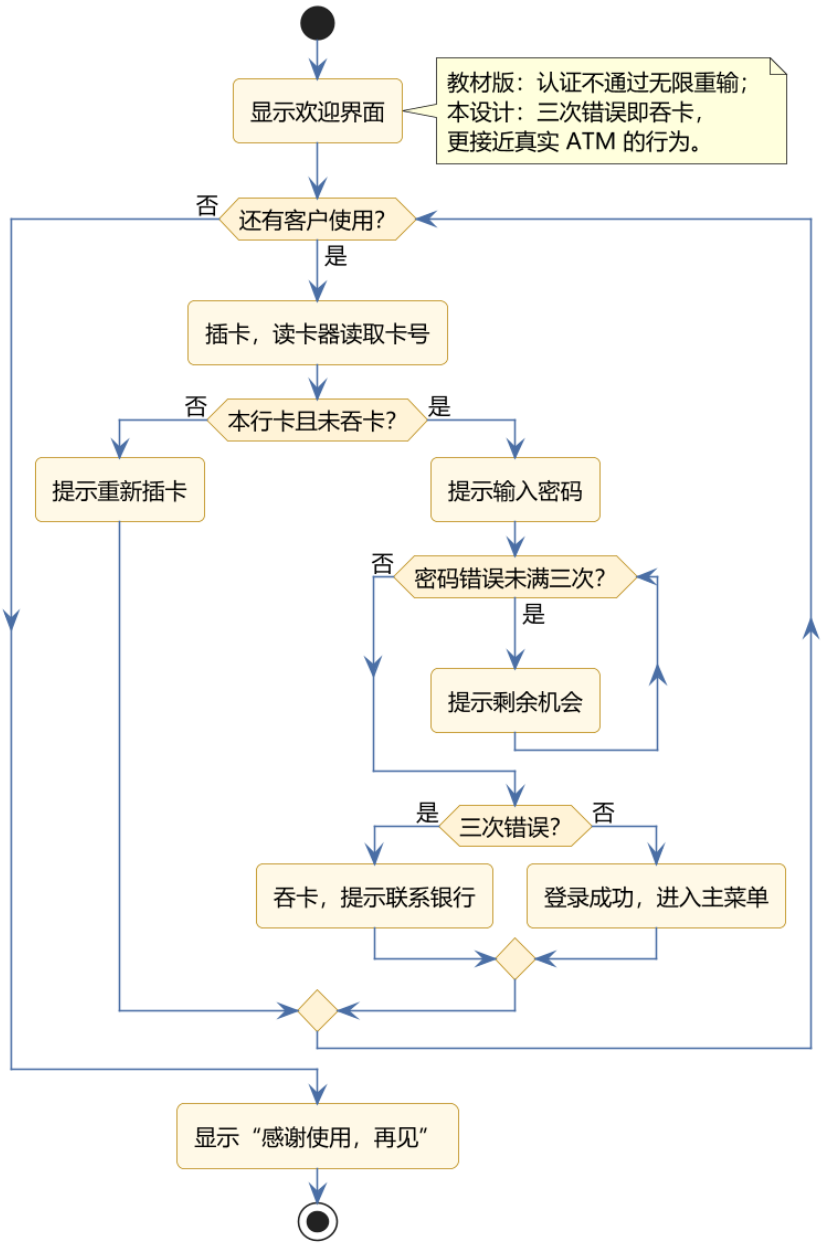


图 5　活动图（一）：插卡与密码认证

认证有一个外层循环（是否继续服务客户）和一个内层循环（密码重试），内层循环最多转 3 圈，第 3 次错误后吞卡、结束本次会话——与课件“无限重输”不同，真实 ATM 必须让重试有上限。

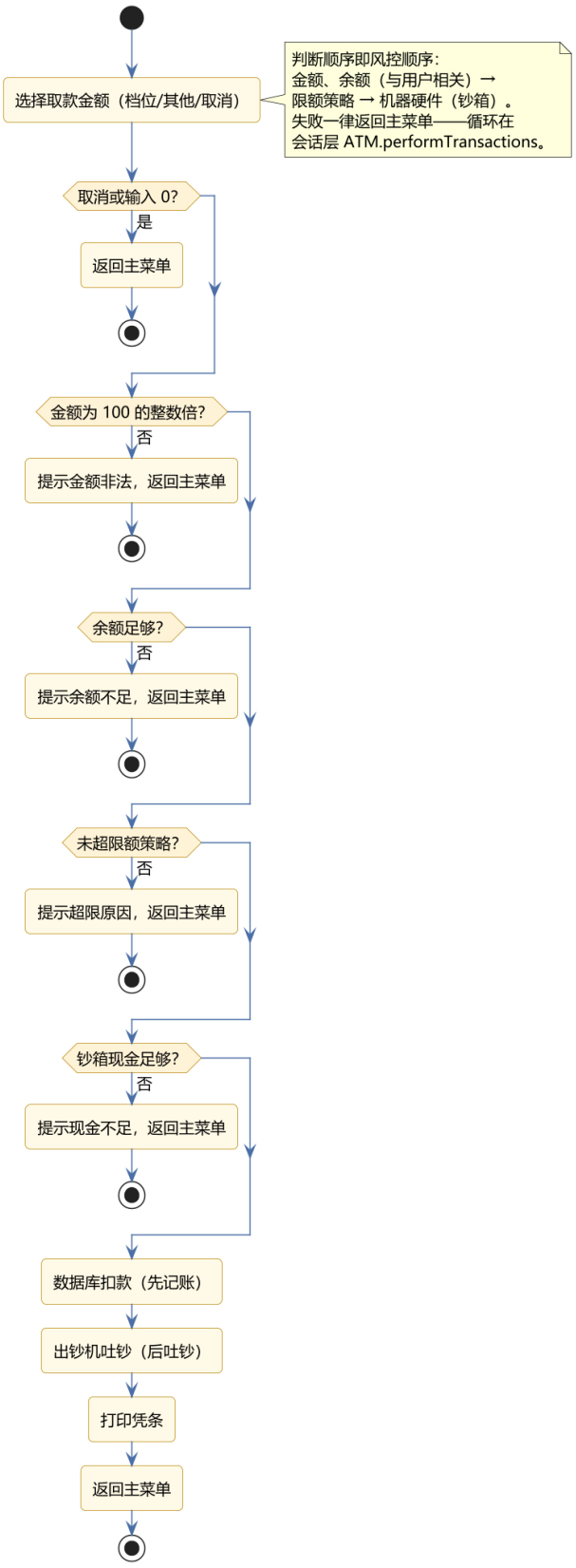


图 6　活动图（二）：取款：四个判断的顺序

取款有四个判断，顺序即风控顺序：先做与用户直接相关、代价低的判断（金额格式、余额），再查限额策略，最后才检查机器硬件（钞箱）。顺序如果反过来，用户会在“自己余额不足”的时候先看到一条与自己无关的“机器没钱”。失败一律直接返回主菜单——循环不在交易内部，而在会话

层的交易主循环里，这也是本设计与课件版活动图（回环画在交易内部）的一个有意差异：交易类保持“一次调用只处理一笔”的无状态语义。

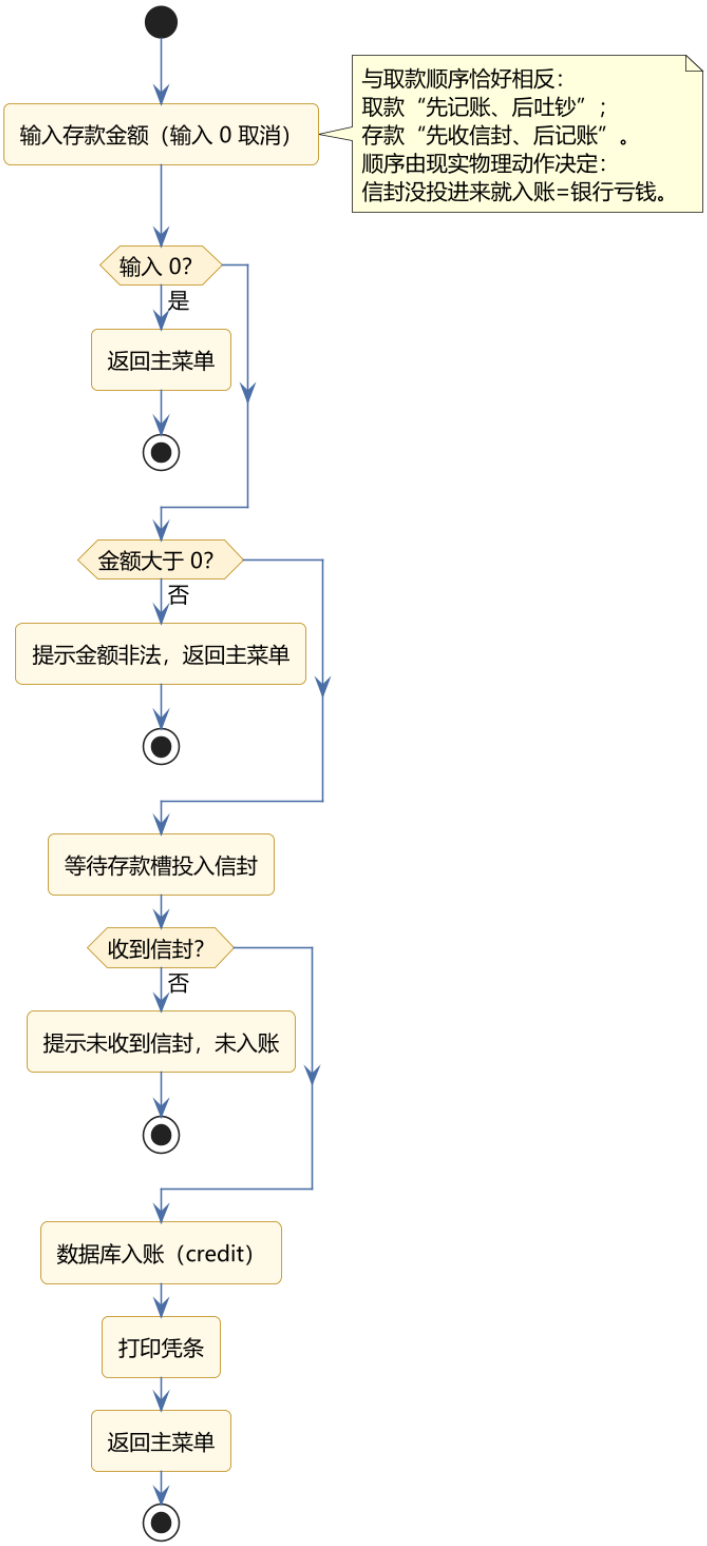


图 7　活动图（三）：存款：先收到信封，后记账

存款与取款的顺序恰好相反：取款是“先记账、后吐钞”，存款是“先收到信封、后记账”。顺序由现实世界的物理动作决定——信封还没投进来就入账，等于银行凭空亏钱。

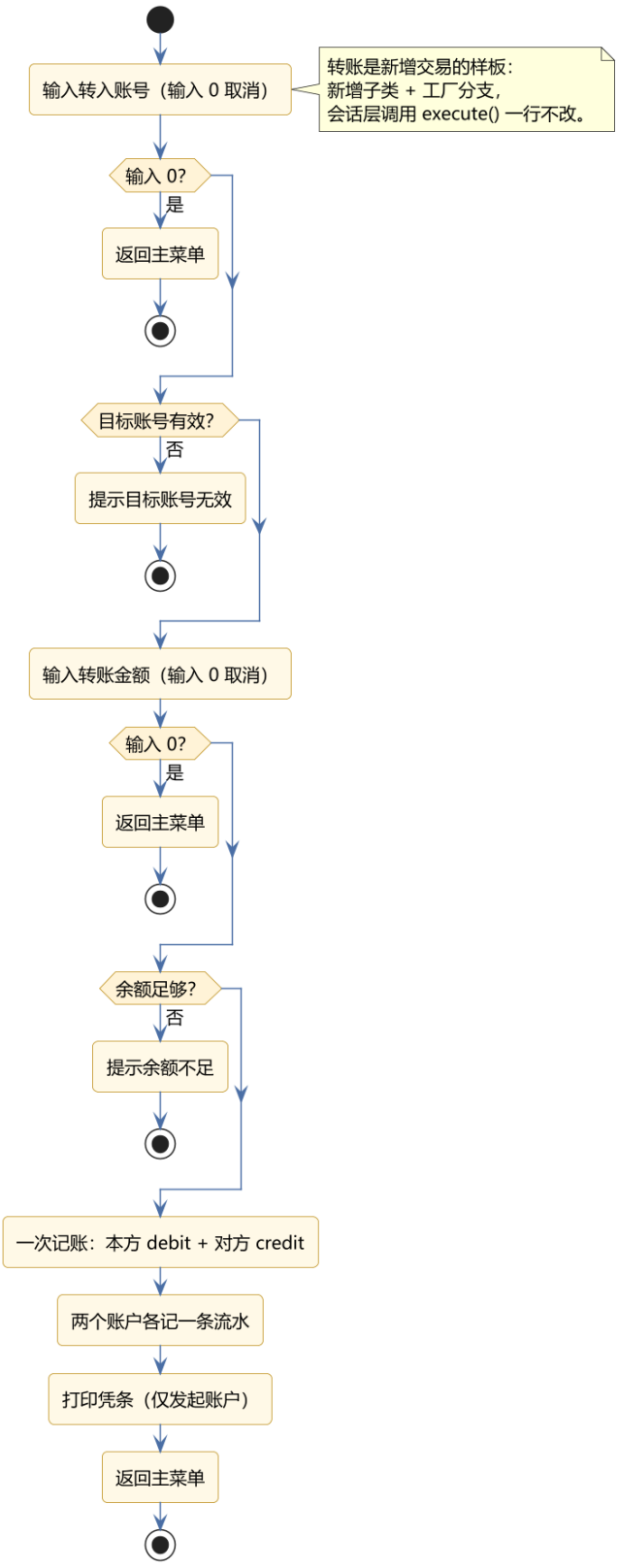


图 8　活动图（四）：转账：多账户协作

转账是最能说明多态价值的用例：它作为新增交易，只需要新增一个子类并在工厂加一个分支，ATM 中调用 execute() 的那行代码完全不需要改动。

2.7 动态结构：时序图

时序图回答“谁调用谁”。一条原则贯穿三张时序图：每条消息的发起者就是这一步的“责任人”。

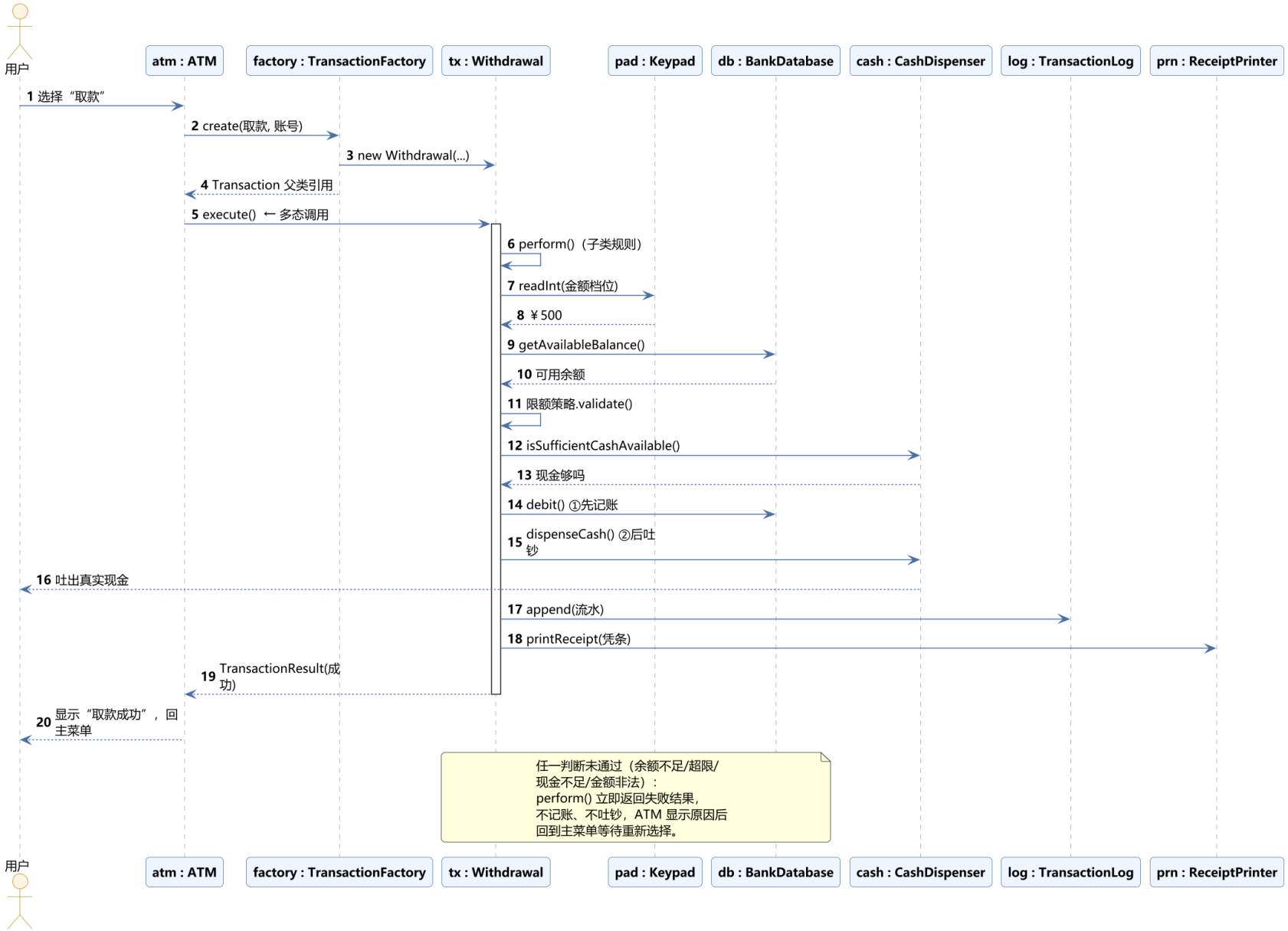


图 9　时序图（一）：取款：工厂、多态调用与模板方法内四重校验

一次取款从会话层开始：ATM 把“要哪种交易”交给工厂，工厂装配出 Withdrawal 对象并返回父类引用；ATM 调用 execute() 时发生动态分派——运行期落实到 Withdrawal 的规则上（第 5 号消息）。execute() 内部先进入 perform() 完成四个判断，然后统一完成记流水与打凭条。失败路径不触发记账与吐钞，直接携带原因返回。

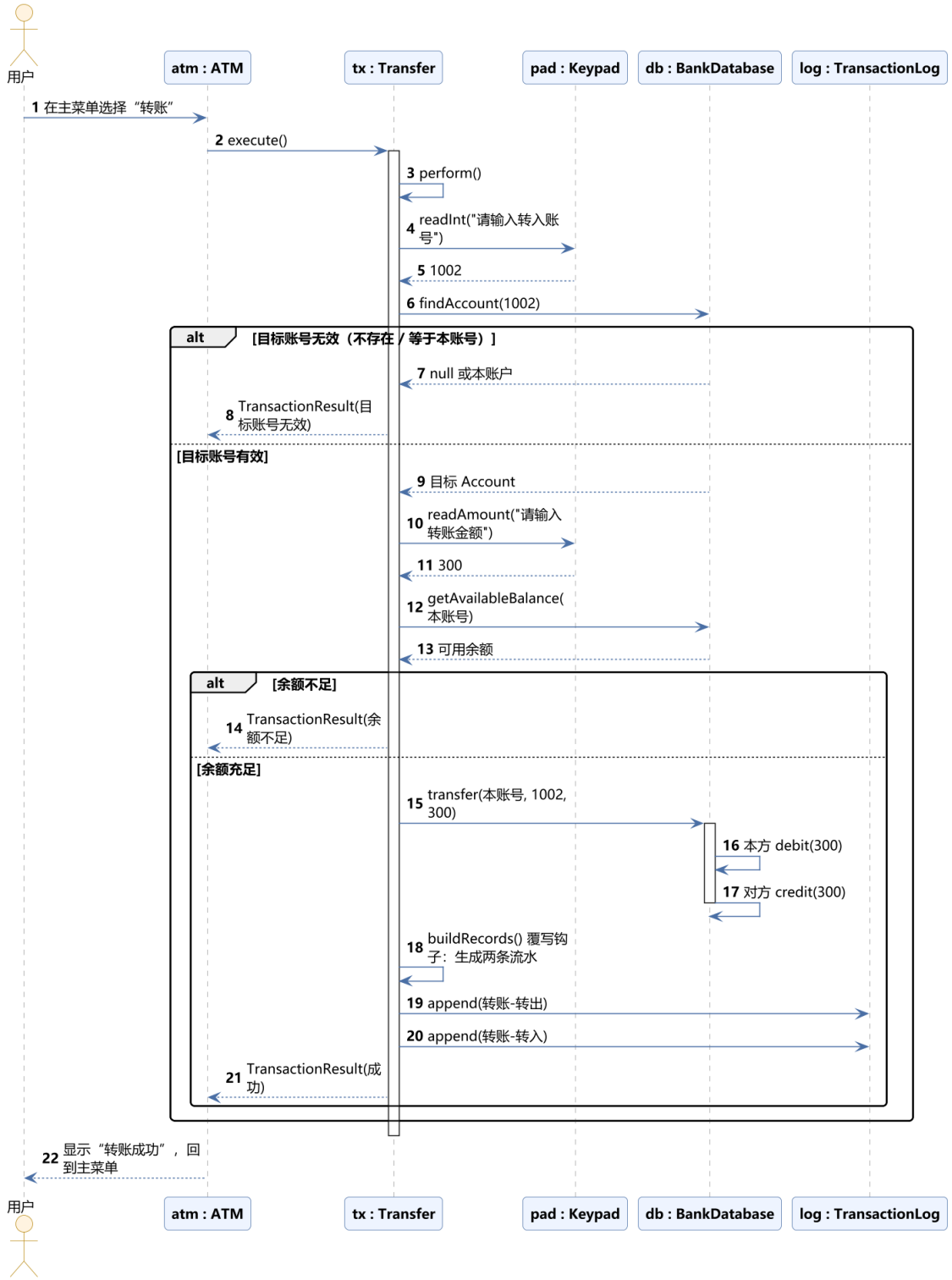


图 10　时序图（二）：转账：一次扣款入账，两条流水

转账比其他交易多两个动作：校验目标账号（查询目标账户）与双边记账（本方 debit + 对方 credit 在一次记账动作内完成）；随后覆写的钩子方法生成两条流水，分别记到转出与转入账户名下。

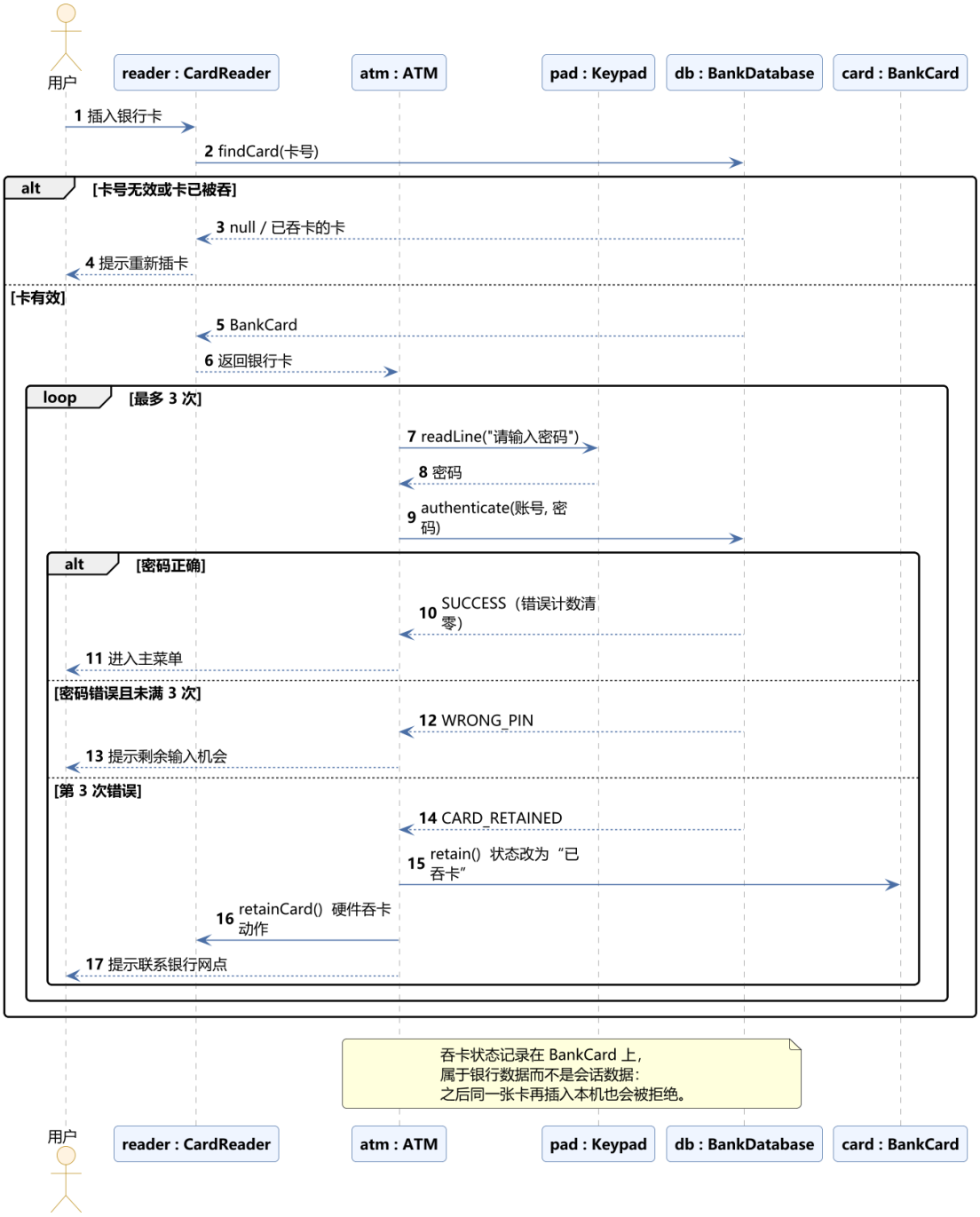


图 11　时序图（三）：插卡认证：循环输入与吞卡分支

认证的循环语义直接体现在时序图上（loop 最多 3 次）：密码正确 → 错误计数清零并进入主菜单；密码错误 → 提示剩余机会；第 3 次错误 → 卡对象状态改为“已吞卡”并执行硬件吞卡动作。注意吞卡状态记录在 BankCard 上——它属于银行数据而不是会话数据，因此同一张卡之后再插入本机也会被拒绝。

2.8 状态建模：银行卡的一生

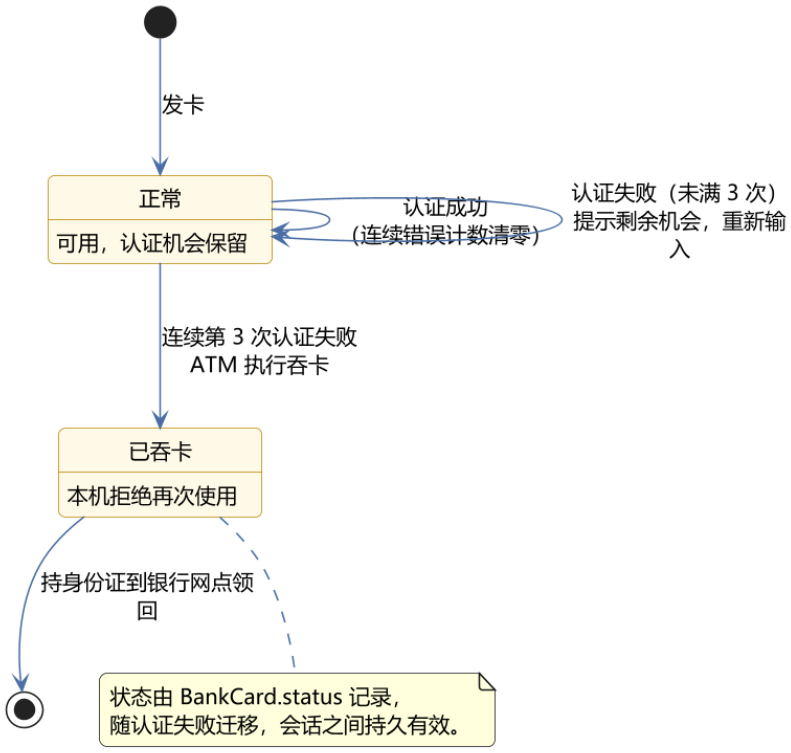


图 12　状态图：银行卡的一生

状态图补足其他视图没有直接表达的信息：卡在两个状态之间迁移。认证成功会把连续错误计数清零（回到“正常”），第 3 次失败则迁移到“已吞卡”，后者只能由银行网点人工处理——这条规则决定了 BankCard 必须是一个有状态的对象而不是一串数字。

2.9 可见性设计

成员	可见性	理由
Account.accountNumber / ownerName	private	只读信息，提供 getter
Account.availableBalance / totalBalance / withdrawnToday	private	余额不允许外部直接改，必须走 debit / credit
Account.pin	private	只提供 checkPin 这一种使用方式
BankDatabase.accounts / cards / transactionLog	private	唯一入口类，外部通过方法访问
BankCard.status	private	由 retain() 单方向迁移
Transaction.accountNumber / bankDatabase / display / receiptPrinter	protected final	子类可用、可读，但一经创建不可修改
Transaction.execute()	public final	对外统一入口，骨架不允许覆盖
Transaction.perform() / getTypeName()	protected / public abstract	子类必答项
Transaction.buildRecords()	protected	钩子方法，允许子类选择性覆写
Withdrawal.keypad / cashDispenser 等子类依赖	private	子类自己的依赖，不对外

三、代码分析

3.1 工程结构与运行方式

工程共 35 个源文件，分为六个包。课件版是 12 个类；多出来的部分主要来自“把硬件抽成接口 + 增加控制台实现”“按包分层”“新增的交易与策略”，每一处都有对应的设计理由。

```
ATM 案例分析/
├── build.bat / run.bat            # 编译、运行脚本（UTF-8 编码处理）
├── src/com/atm/
│   ├── ATM.java                   # 会话主控：插卡→认证→交易循环→退卡
│   ├── AtmApplication.java        # 组合根：装配硬件实现并注入 ATM
│   ├── bank/                      # 银行数据层
│   │   ├── BankDatabase.java      #   数据唯一入口（饿汉单例）
│   │   ├── Account.java / BankCard.java
│   │   ├── TransactionLog.java / TransactionRecord.java
│   │   └── AuthResult.java / CardStatus.java
│   ├── policy/                    # 取款限额策略
│   │   ├── WithdrawalLimitPolicy.java
│   │   ├── AbstractLimitPolicy.java
│   │   └── StandardLimitPolicy.java / VipLimitPolicy.java
│   ├── transaction/               # 交易家族
│   │   ├── Transaction.java       #   sealed 抽象父类（模板方法）
│   │   ├── BalanceInquiry.java / HistoryInquiry.java
│   │   ├── Withdrawal.java / Deposit.java / Transfer.java
│   │   ├── TransactionResult.java #   结果记录类
│   │   └── TransactionFactory.java
│   ├── hardware/                  # 硬件接口（6 个）
│   │   └── console/               # 控制台实现（6 个）
│   └── ui/
│       ├── Menu.java              # 泛型菜单组件
│       └── MainMenuChoice.java
└── test/
    ├── SmokeTest.java             # 冒烟测试（55 项断言，测试替身）
    └── session-*.txt / out-*.txt  # 端到端输入脚本与输出记录
```

包	内容	面向的变化
bank	账户、卡、流水、数据库	数据来源变化（内存 → 数据库）
policy	限额策略接口与实现	银行规则变化（限额、VIP 政策）
transaction	交易父类与五个子类	业务种类变化（新增交易）
hardware + hardware.console	硬件接口与控制台实现	设备变化（控制台 → 真实设备 / 图形界面）
ui	菜单组件、菜单选项	界面形式变化
（根）ATM、AtmApplication	流程编排与装配	启动方式变化

编译与运行（源码为 UTF-8，编译必须带 -encoding UTF-8）：

```
build.bat                      # javac -encoding UTF-8 -d out -sourcepath src
src\com\atm\AtmApplication.java
run.bat                        # java -Dstdout.encoding=UTF-8 -cp out com.atm.AtmApplication
java -cp "out;out-test" SmokeTest         # 冒烟测试
java -cp out com.atm.AtmApplication < test\session-main.txt   # 端到端脚本
```

3.2 继承与多态的应用

3.2.1 抽象父类：公共数据上提，骨架固定

```
public sealed abstract class Transaction
        permits BalanceInquiry, Deposit, HistoryInquiry, Transfer, Withdrawal {

    private final int accountNumber;
    private final BankDatabase bankDatabase;
    private final Display display;
    private final ReceiptPrinter receiptPrinter;

    protected Transaction(int accountNumber, BankDatabase bankDatabase,
                          Display display, ReceiptPrinter receiptPrinter) {
        this.accountNumber = accountNumber;
        this.bankDatabase = bankDatabase;
        this.display = display;
        this.receiptPrinter = receiptPrinter;
    }
```

sealed 使交易家族封闭；四个依赖用构造器注入（子类不自己 new 任何东西）；字段 private final，交易对象一旦建好即不可变。

```
    public final TransactionResult execute() {
        display.showMessageLine("");
        display.showMessageLine("—— " + getTypeName() + " ——");

        TransactionResult result = perform();

        // 模板固定的一步：按结果展示文字
        if (result.isSuccess()) {
            display.showMessageLine(result.message());
        } else if (result.isCancelled()) {
            display.showMessageLine("已取消本次" + getTypeName() + "。");
        } else {
            display.showMessageLine("交易未完成：" + result.message());
        }

        // 模板固定的一步：记流水（钩子可扩展）＋成功时打凭条
        List<TransactionRecord> records = buildRecords(result);
        for (TransactionRecord record : records) {
            bankDatabase.getTransactionLog().append(record);
        }
        if (result.isSuccess()) {
            receiptPrinter.printReceipt(records.get(0));
        }
        return result;
    }
```

execute() 是 final 模板方法：它把“执行规则（perform）→ 按结果展示 → 记流水 → 打凭条”固化为骨架。子类不需要、也无法改动这个骨架——记账之后必然有流水，成功之后必然打印凭条，这类“规矩”由父类保证。

```
    protected abstract TransactionResult perform();

    /** 交易类型名，用于菜单、流水与凭条的显示。 */
    public abstract String getTypeName();
```

perform() 与 getTypeName() 是子类的必答项：五类交易的行为差异全部收敛在这两个方法里。

3.2.2 子类只写规则：取款的四重判断

```
        // 判断一：金额本身合法吗（正数、100 的整数倍）
        if (amount < 0 || Math.abs(amount % CashDispenser.BILL_VALUE) > 1e-9) {
            return TransactionResult.failure(TransactionResult.Status.INVALID_AMOUNT,
                    "取款金额必须为 100 元的整数倍。");
        }

        // 判断二：账户余额够吗
        double available = getBankDatabase().getAvailableBalance(getAccountNumber());
        if (amount > available) {
            return TransactionResult.failure(TransactionResult.Status.INSUFFICIENT_BALANCE,
                    String.format("账户余额不足（可用余额 ¥%,.2f），请重新选择取款金额。",
available));
        }

        // 判断三：限额策略允许吗
        Account account = getBankDatabase().findAccount(getAccountNumber());
        Optional<String> limitViolation = account.getLimitPolicy()
                .validate(amount, account.getWithdrawnToday());
        if (limitViolation.isPresent()) {
            return TransactionResult.failure(TransactionResult.Status.LIMIT_EXCEEDED,
                    limitViolation.get());
        }

        // 判断四：钞箱里的现金够吗
        if (!cashDispenser.isSufficientCashAvailable(amount)) {
            return TransactionResult.failure(TransactionResult.Status.INSUFFICIENT_CASH,
                    "取款机现金不足，请选择更小的取款金额。");
        }

        // 先记账、再吐钞
        getBankDatabase().debit(getAccountNumber(), amount);
        getBankDatabase().registerWithdrawal(getAccountNumber(), amount);
        cashDispenser.dispenseCash(amount);

        return TransactionResult.success(
                "取款成功。",
                amount, getBankDatabase().getTotalBalance(getAccountNumber()));
    }
}
```

这段代码就是活动图（图 6）的直接翻译：四个判断的顺序与图完全一致。两点值得注意：其一，每个判断都“先检查、再动手”，任何一步不通过都不会触碰数据库与硬件；其二，记账（debit）在吐钞（dispenseCash）之前——账已扣、钞未出还能人工对账；反过来钱出去了账没扣，则再也追不回来。顺序即风控，这是业务代码的常识。

3.2.3 会话层的多态调用点

```
            // 多态：声明类型是父类 Transaction，实际执行哪个子类的方法由运行时对象决定
            Transaction transaction = transactionFactory.create(choice, accountNumber);
            transaction.execute();
            // 成功、失败、取消都回到主菜单，由用户决定下一步
```

这是整个系统中唯一调用 execute() 的地方，也是多态的唯一入口：transaction 的声明类型永远是父类 Transaction，实际执行哪段规则由运行时对象的实际类型决定。编译期按父类检查（只能调用父类声明的方法），运行期按实际对象分派（执行子类重写的版本）。把交易种类从 5 种扩展到 50 种，这一行代码一个字都不用改——这就是多态的价值。

3.2.4 钩子方法：转账的双流水

```
    protected List<TransactionRecord> buildRecords(TransactionResult result) {
        if (!result.isSuccess() || targetAccountNumber < 0) {
            return super.buildRecords(result);
        }
        LocalDateTime now = LocalDateTime.now();
        TransactionRecord outRecord = new TransactionRecord(now, getAccountNumber(),
                "转账-转出", result.amount(),
                getBankDatabase().getTotalBalance(getAccountNumber()),
result.status().getLabel());
        TransactionRecord inRecord = new TransactionRecord(now, targetAccountNumber,
                "转账-转入", result.amount(),
                getBankDatabase().getTotalBalance(targetAccountNumber),
result.status().getLabel());
        return List.of(outRecord, inRecord);
    }
```

模板方法把“记流水”设计为可覆写的钩子：默认实现（父类）为当前账户记一条；转账需要给两个账户各记一条，于是子类覆写钩子。骨架没有改变，变化被约束在一个方法里。

3.2.5 依赖注入与方法引用

```
        CardReader cardReader = new ConsoleCardReader(display, keypad,
bankDatabase::findCard);
```

这是入口装配读卡器的一行：查卡动作以 bankDatabase::findCard（方法引用，等价于一个 IntFunction<BankCard>）注入读卡器。读卡器因此不需要知道银行数据库的存在——硬件层与银行层解耦；测试时传入一个假函数即可完全脱离数据库测试读卡逻辑。

3.3 使用的设计模式

模式	本项目落点	解决的问题
模板方法	Transaction.execute()（final）＋ perform() / buildRecords()	统一执行骨架，子类只写规则；公共步骤不重复
策略	WithdrawalLimitPolicy 及其实现；Account 持有策略	取款限额规则可整体替换；新增客户类型不改交易代码
简单工厂	TransactionFactory.create()	把“菜单选择 → 具体交易对象”的装配逻辑集中一处
单例	BankDatabase（饿汉式）	账户数据的唯一入口，避免多处持有副本
命令（影子）	Transaction 对象本身即“一次交易请求”	ATM 是请求发起方（invoker），BankDatabase 是接收方（receiver）

关于每个模式的几句补充：

- 模板方法模式若不使用：三类交易的 execute() 各自实现，则“记流水、打凭条、结果展示”三段代码要重复五份，日后加一个公共步骤（比如审计日志）要改五处，漏一处就是事故。
- 策略模式若不使用：限额规则会以 if-else 的形式写死在 Withdrawal 里；而“普通账户、VIP 账户、未来的学生账户”是典型的会不断增生的规则，必须抽成对象才能不改既有代码地扩展。AbstractLimitPolicy 又把“单笔 + 当日累计”两级校验的公共骨架沉到抽象类，两个具体策略只差两组数值。
- 简单工厂若不使用：会话层要认识全部五个交易子类的构造细节（每个子类依赖的硬件不同），会退化成 switch 里几十行的装配代码，且每新增交易都要改会话层。
- 单例若不使用：数据入口在多处被创建，余额状态会产生副本，对账将失去意义。

3.4 关键实现与设计的一致性

- 活动图即代码结构：取款活动图里的四个菱形，对应 Withdrawal.perform() 里四段 if；存款、转账同样一一对应。图纸先行的一个好处是：编码阶段几乎没有出现“逻辑顺序怎么排”的犹豫。
- 顺序即业务：先记账后吐钞、先收信封后记账，两个相反的顺序都由现实约束决定，而不是随意选择。
- 结果对象统一处理：成功、失败、取消三种结局全部封装为 TransactionResult（含状态与原因），模板方法中三行 switch 统一展示，流水与凭条从同一份数据生成，不存在“某条路径忘记记流水”的可能。
- 输入回显：键盘在读取后把输入原样回显到屏幕，使“脚本化输入跑出的记录”与“人工操作看到的画面”完全一致——这是端到端自动化验证能成立的前提。

3.5 测试与验证

业务层自动测试（SmokeTest）：用“测试替身”替换全部六个硬件（脚本化键盘、记录型屏幕、假钞箱、假存款槽、假凭条机），不产生任何真实输入输出，共 55 项断言，覆盖认证（含三次吞卡）、余额查询、取款四类失败与成功、存款信封分支、转账五类分支、流水查询、菜单非法输入重试等；全部通过。这组测试本身就是“硬件面向接口”这一设计的直接收益：没有接口，业务规则测试将不得不依赖控制台交互。

端到端会话脚本：把输入重定向为脚本文件，完整跑通四个场景，输出重定向到文件后逐项检查关键提示与退出码：

脚本	场景	主要覆盖点	结果
session-main.txt	完整业务主流程	查询、取款（失败重试 + 成功）、存款、转账、流水、退出	通过（退出码 0，无异常）
session-retain.txt	认证与吞卡	未知卡号拒收、三次密码错误吞卡、已吞卡卡再次插入被拒	通过
session-limit.txt	规则边界	超日限额、恰好到限额、取款取消、转账目标无效 / 本账户 / 余额不足、存款未投信封	通过

session-vip.txt	VIP 与钞箱	超单笔限额场景下钞箱不足、成功大额取款、余额与流水变化	通过

开发过程中发现并修复的问题（如实记录）：① 主菜单链式初始化时泛型推断失败（Menu<Object>），在编码阶段即修复——改为显式类型参数；② 取款成功提示与出钞机提示重复播报金额，已简化为单一来源；③ 首版端到端脚本把“取款菜单层”的输入写成了“主菜单层”的输入，导致会话错位——程序行为本身正确，脚本按设计修正后通过；此事也说明端到端脚本必须与菜单结构逐层对齐。

3.6 优点与缺点

优点	说明
分层清晰、职责单一	每个类只有一个变化原因；会话、规则、数据、输入输出四类职责互不越界
扩展点明确	新增交易＝1 个子类 + 1 个工厂分支（多态保证调用点不动）；新增限额＝1 个策略类
可测试	全部硬件面向接口，55 项业务断言零真实 IO；端到端脚本可重复执行
设计闭环	用例、类图、活动图、时序图、状态图与代码一一对应，图纸即文档
现代 Java 特性用得有理由	sealed 封闭交易家族；record 表达不可变结果与流水；enum 状态自带中文标签；泛型菜单消除重复
失败路径完整	8 种失败状态全部有明确分支、提示与流水记录；不会出现“沉默失败”

缺点	说明
金额用 double	存在二进制浮点误差隐患；生产必须用 BigDecimal 或“分”整数。这是本设计最需要改正的地方
无持久化	数据在内存，进程退出即丢失；吞卡、限额等状态不跨进程，转账也没有数据库事务保证
“当日已取”未绑定真实日期	演示数据预置了当日已取金额，但没有实现跨业务日清零的逻辑
有限并发假定	按单机单会话设计；多终端共享同一账户时的并发控制（乐观锁 / 事务）未涉及
Transfer 的状态纯度	targetAccountNumber 是 perform() 写入、钩子读取的可变字段；可以用“返回结果携带目标账户”的方式重构得更纯粹
界面与文案硬编码	表格宽度、对齐、金额格式散落在控制台实现中；多语言与界面切换需要抽取
无日志文件	流水只在内存中；审计所需的持久日志尚未实现

3.7 改进方向

按“生产可用”的优先级，我们认为本设计可以沿以下方向改进：

1. 金额类型：引入 BigDecimal（或统一“分”为单位的长整数），消除浮点误差；格式化集中到一个 Money 工具类。

2. 持久化与事务：用 JDBC 落地 Account、Card、流水三张表；转账、吞卡用数据库事务保证原子性；BankDatabase 改为 DAO 外观，接口保持不变——这正是分层设计预留的替换点。

3. 业务日翻转：每日限额按业务日重置（数据库字段 + 定时任务或首次访问时惰性重置）。

4. ATM 状态机：把“欢迎 / 读卡 / 认证 / 交易 / 退卡”抽象为显式状态机，适配真实设备冗长的状态迁移。

5. 策略组合：用组合策略（单笔 + 日累计 + 月累计 + 渠道限额）替代现在的两级校验，策略之间可叠加。

6. 图形界面：实现一套 Swing / JavaFX 版的六个硬件接口，替换 hardware.console 包——入口装配处改六行，其余代码不动。

7. 日志与审计：流水落盘（追加写文件或数据库），增加操作日志与对账任务。

8. 国际化与远程化：提示文案迁移到 ResourceBundle；把 BankDatabase 抽成远程服务（RMI / HTTP），ATM 变为客户端，验证“接口不变、实现可换”的设计承诺。

四、总结

4.1 本次作业完成的内容

我们以第 5 章 ATM 案例为起点，独立完成了从需求分析、面向对象设计到编码与验证的全过程：六个用例的功能需求；13 个类的领域设计（银行层 7 个、策略层 4 个、交易层 8 个、会话层 2 个、界面组件 3 个、硬件接口与实现 12 个，共 35 个源文件）；用例图、3 张类图、4 张活动图、3 张时序图、1 张状态图共 12 张图；以及一套可运行的 Java 实现（55 项自动断言 + 4 组端到端会话脚本全部通过）。

4.2 收获与体会

- 第 5 章的知识点在本项目里全部找到了各自的“岗位”：抽象类与模板方法负责固定骨架（Transaction）；接口负责解耦边界（六个硬件）；多态负责扩展点（execute() 调用位）；泛型负责消除重复（Menu<T>）；sealed 负责家族治理；record 负责不可变数据（TransactionResult、TransactionRecord）；enum 负责有限取值（状态与结果）。
- 设计决定代码的形态：活动图先画清楚，代码里的判断顺序就不再是拍脑袋；时序图先想清楚“谁负责”，方法归属就不会写错地方。图纸不是文档装饰，而是编码的依据。
- “顺序即业务”是本次最大的心得：先记账后吐钞、先收信封后记账、先查余额后查钞箱——同一个系统里三个相反的顺序，每一个都有明确的现实理由。业务代码的正确性往往就藏在这些顺序里。

- 多态的价值只有在“扩展”发生时才真正显现：新增转账交易时，会话层、活动图之外的代码零改动；这也是评价一个设计好坏最直接的标准——变化来了，你只需要改“该改的地方”。
- 面向接口不只是“为了好看”：正因为它，我们才能用 55 项零 IO 的自动化断言验证业务规则，也才能在将来把控制台换成图形界面而不动业务代码。

4.3 不足与展望

受课堂作业的范围限制，本实现仍是“教学级”的：金额精度、持久化、并发、日志、国际化都做了简化（详见 3.6、3.7）。如果继续完善，我们会优先做三件事：把金额换成 BigDecimal、把数据落到 JDBC 并给转账加上事务、把 ATM 会话重构成显式的状态机。到那时，这套设计就不仅是一个“能跑通的案例”，而是一个可以真正交给银行柜台讨论的雏形。

附录

附录 A 端到端会话记录（节选）

场景一：认证与吞卡（test/session-retain.txt 的完整输出）——未知卡号被拒收、三次密码错误吞卡、已吞卡再次插入被拒：

```
========================================
   欢迎使用 Java 银行自动柜员机（ATM）
========================================
请插入银行卡（输入卡号；输入 0 不插卡并退出系统）：77777
无法识别的卡号，请重新插卡。
请插入银行卡（输入卡号；输入 0 不插卡并退出系统）：62220001
（读卡器）正在读取银行卡信息……
请输入密码：000000
密码错误，您还有 2 次机会。
请输入密码：111111
密码错误，您还有 1 次机会。
请输入密码：222222
（读卡器）银行卡已收入吞卡槽。
密码连续输错次数过多，银行卡已被吞卡，请联系银行网点处理。
请插入银行卡（输入卡号；输入 0 不插卡并退出系统）：62220001
该卡已因密码错误过多被本机吞卡，无法使用，请联系银行网点。
请插入银行卡（输入卡号；输入 0 不插卡并退出系统）：0

感谢使用，再见！
```

场景二：主流程中的取款片段（test/session-main.txt 输出节选）——取款成功、记账、吐钞与凭条：

```
—— 取款 ——

【取款菜单】
1 - ￥100
2 - ￥200
3 - ￥500
4 - ￥1000
5 - ￥2000
6 - 其他金额
```

```
7 - 取消交易
请输入您的选择：abc
输入无效，请输入数字。
请输入您的选择：2
（出钞口）请取走您的现金：￥200.00，共 2 张 100 元。
取款成功。

        — 交易凭条 —
时间：2026-10-13 18:32:07
账号：1001
类型：取款
金额：￥200.00
余额：￥1,000.00
结果：成功
（凭条打印完成）
```

附录 B 编译、运行与测试

```
:: 编译（源码 UTF-8，必须带 -encoding UTF-8）
build.bat

:: 运行
run.bat

:: 冒烟测试（55 项断言）
javac -encoding UTF-8 -d out-test -cp out test\SmokeTest.java
java -Dstdout.encoding=UTF-8 -cp "out;out-test" SmokeTest

:: 端到端脚本（把输入重定向为文件即可复现会话）
java -Dstdout.encoding=UTF-8 -cp out com.atm.AtmApplication < test\session-main.txt
```